
HELIOSAT API

Real-Time Space-Weather Forecast

Client Integration Manual

API version: v1 **Document:** June 8, 2026

Base URL: <https://heliosat-live.vercel.app>

Authentication: API key (HTTP Authorization: Bearer)

Support: contact@heliosat.co

Contents

1	Overview	2
1.1	Key concepts	2
2	Quick start	2
3	Authentication	2
4	Endpoints	3
4.1	Real-time forecast	3
4.2	Service status	3
5	Response schema (v1)	3
5.1	Example response	3
5.2	Top-level fields	4
5.3	observed object	4
5.4	arrival object	4
5.5	inbound_peak object	4
6	Status & error codes	5
7	Rate limiting	5
8	Code examples	5
8.1	cURL	5
8.2	Python	5
8.3	JavaScript (Node 18+)	6
8.4	Handling rate limits (Python)	6
9	Versioning & stability	6
10	Support	6

1 Overview

The HELIOSAT API delivers a **real-time space-weather nowcast** as a single, versioned JSON resource. It reports the solar wind currently measured at the L1 Lagrange point, propagated to its estimated Earth arrival, together with the resulting geomagnetic activity level.

- **One endpoint, one job.** GET `/api/v1/forecast/realtime` returns the latest forecast. No heavy queries, no parameters required.
- **Always fresh.** The forecast is precomputed server-side and refreshed roughly every 5 minutes, so every request reads a ready value and responds in tens of milliseconds.
- **Stable contract.** Within `v1`, the response shape only ever grows (new fields); existing fields are never removed or changed.

1.1 Key concepts

L1

The Sun–Earth Lagrange point (~ 1.5 million km sunward of Earth) where solar-wind monitors sit. Conditions seen at L1 reach Earth tens of minutes later.

Bz (nT)

North–south component of the interplanetary magnetic field. *Negative (southward) Bz* couples energy into Earth’s magnetosphere and drives geomagnetic storms.

G-scale

NOAA’s geomagnetic storm scale, 0 (quiet) to 5 (extreme).

Inbound parcel

Solar wind already past L1 but not yet at Earth — i.e. weather that *will* arrive within the lead time.

2 Quick start

1. Obtain your API key from HELIOSAT (see §3).
2. Make one authenticated request:

```
curl -H "Authorization: Bearer hsk_live_XXXXXXXXXXXXXXXXX" \  
      https://heliosat-live.vercel.app/api/v1/forecast/realtime
```

A 200 OK returns the JSON described in §5.

3 Authentication

Every request must include your API key as a bearer token:

```
Authorization: Bearer <your_api_key>
```

- Keys look like `hsk_live_...` and are issued per client/environment.
- Treat the key as a secret: send it only from your backend, never embed it in a browser, mobile app, or public repository.
- HELIOSAT stores only a one-way hash of your key; it cannot be recovered. If a key is lost or leaked, request a new one and the old one is revoked.

- A missing or invalid key returns 401 (see §6).

4 Endpoints

4.1 Real-time forecast

GET `https://heliosat-live.vercel.app/api/v1/forecast/realtime`

Returns the latest precomputed forecast. Requires authentication. Responses are not cached (`Cache-Control: no-store`); poll as often as your rate limit allows (refreshes land about every 5 minutes, so polling faster rarely yields new data).

4.2 Service status

GET `https://heliosat-live.vercel.app/api/v1/status`

A lightweight, *unauthenticated* health check. Returns 200 when a fresh forecast is available and 503 when the data is stale or unavailable — convenient for an uptime monitor.

```
{
  "status": "ok",
  "schema_version": "1",
  "issued_at": "2026-06-07T23:28:29.531Z",
  "forecast_age_seconds": 60,
  "stale": false
}
```

5 Response schema (v1)

Units & conventions. Speed in kms^{-1} ; magnetic field in nT; density in particles cm^{-3} ; all timestamps are ISO-8601 in UTC. Any of `observed`, `arrival`, and `inbound_peak` may be null when the corresponding data is unavailable. **Clients must ignore unknown fields.**

5.1 Example response

```
{
  "schema_version": "1",
  "issued_at": "2026-06-07T23:28:29.531Z",
  "observed_at": "2026-06-07T23:25:00.000Z",
  "l1_distance_km": 1590297.74,
  "observed": {
    "speed_km_s": 521.3,
    "bz_nt": 3.69,
    "density_p_cm3": 1.66,
    "g_level": 0
  },
  "arrival": {
    "estimated_utc": "2026-06-08T00:15:50.638Z",
    "transit_lag_minutes": 51
  },
  "inbound_peak": {
    "g_level": 0,
    "speed_km_s": 560,
    "min_bz_nt": 3,
    "eta_utc": "2026-06-07T23:28:46.951Z",
    "lead_minutes": 47
  }
}
```

```
}
}
```

5.2 Top-level fields

Field	Type	Description
schema_version	string	Contract version of this payload. Currently "1".
issued_at	string	When this forecast was published (the precompute run time).
observed_at	string null	Timestamp of the latest L1 measurement used.
l1_distance_km	number	L1 distance (km) used to propagate the wind to Earth.
observed	object null	Latest measured solar-wind conditions (Table 2).
arrival	object null	Earth-arrival estimate for the latest parcel (Table 3).
inbound_peak	object null	Worst parcel still inbound, or null (Table 4).

Table 1: Top-level response object.

5.3 observed object

Field	Type	Description
speed_km_s	number null	Solar-wind bulk speed (km s^{-1}).
bz_nt	number null	IMF Bz, GSM (nT). Negative = southward = geoeffective.
density_p_cm3	number null	Proton density (particles cm^{-3}).
g_level	integer	NOAA G-scale right now (0–5).

Table 2: **observed** — current conditions at L1.

5.4 arrival object

Field	Type	Description
estimated_utc	string null	Estimated Earth-arrival time of the latest parcel.
transit_lag_minutes	number null	L1 → Earth travel time (minutes).

Table 3: **arrival** — when the latest measured parcel reaches Earth.

5.5 inbound_peak object

Field	Type	Description
g_level	integer	Peak NOAA G-scale among the inbound parcels.
speed_km_s	number	Peak speed among the inbound parcels (km s^{-1}).
min_bz_nt	number	Most negative Bz among the inbound parcels (nT).
eta_utc	string	Estimated Earth-arrival of the worst inbound parcel.
lead_minutes	integer	Minutes from issue time until the inbound window arrives.

Table 4: **inbound_peak** — strongest weather still in transit.

6 Status & error codes

HTTP	Meaning	Notes
200	OK	Forecast (or status) returned.
401	Unauthorized	Missing, invalid, inactive, or expired API key. Header <code>WWW-Authenticate: Bearer.</code>
429	Too Many Requests	Rate limit exceeded. See <code>Retry-After</code> .
503	Service Unavailable	No forecast available yet, or temporarily unconfigured. Retry shortly.

Table 5: Response codes.

Error bodies are JSON: `{ "error": "human-readable message" }`.

7 Rate limiting

Requests are limited **per key, per minute**. Every response carries the current budget; 429 responses also tell you when to retry.

Header	Meaning
<code>X-RateLimit-Limit</code>	Maximum requests allowed per minute for your key.
<code>X-RateLimit-Remaining</code>	Requests remaining in the current 60-second window.
<code>Retry-After</code>	(On 429) seconds until the window resets.

Table 6: Rate-limit headers.

Best practice. Cache the last good forecast on your side and poll no faster than you need (the data changes about every 5 minutes). On 429, wait for `Retry-After` seconds before retrying.

8 Code examples

8.1 cURL

```
curl -sS -H "Authorization: Bearer hsk_live_XXXXXXXXXXXXXXXXX" \
  https://heliosat-live.vercel.app/api/v1/forecast/realtime
```

8.2 Python

```
import requests

API_KEY = "hsk_live_XXXXXXXXXXXXXXXXX"
BASE_URL = "https://heliosat-live.vercel.app"

resp = requests.get(
    f"{BASE_URL}/api/v1/forecast/realtime",
    headers={"Authorization": f"Bearer {API_KEY}"},
    timeout=10,
)
resp.raise_for_status()
forecast = resp.json()
```

```
obs = forecast["observed"]
print(f"Speed {obs['speed_km_s']} km/s, Bz {obs['bz_nt']} nT, G{obs['g_level']}")
```

8.3 JavaScript (Node 18+)

```
const API_KEY = "hsk_live_XXXXXXXXXXXXXXXXX";
const BASE_URL = "https://heliosat-live.vercel.app";

const res = await fetch(`${BASE_URL}/api/v1/forecast/realtime`, {
  headers: { Authorization: `Bearer ${API_KEY}` },
});
if (!res.ok) throw new Error(`HELIOSAT API error: ${res.status}`);
const forecast = await res.json();
console.log(forecast.observed.speed_km_s, forecast.observed.g_level);
```

8.4 Handling rate limits (Python)

```
import time, requests

def get_forecast(api_key, base_url, retries=3):
    headers = {"Authorization": f"Bearer {api_key}"}
    for _ in range(retries):
        r = requests.get(f"{base_url}/api/v1/forecast/realtime",
                        headers=headers, timeout=10)
        if r.status_code == 429:
            time.sleep(int(r.headers.get("Retry-After", "5")))
            continue
        r.raise_for_status()
        return r.json()
    raise RuntimeError("Rate limited; please retry later.")
```

9 Versioning & stability

- The major version lives in the path (`/api/v1/...`) and is echoed in `schema_version`.
- Within a major version, changes are **additive only**: new fields may appear; existing fields are never renamed, removed, or retyped. *Always ignore fields you do not recognise.*
- A breaking change ships as a new major version under a new path (`/api/v2/...`); the previous version remains available during an announced deprecation window.
- Deprecations are communicated in advance to active keys.

10 Support

Questions, key requests, or higher rate limits: contact@heliosat.co.